

DSA × AI Engineering

Learn by Building — Full Curriculum

Week 1 → Week 4 • Day-by-Day Playbook

Bobby · Lead AI Engineer · Mahindra

■ Week 1	Arrays, HashMaps, Stacks, Queues, Linked Lists
■ Week 2	Embeddings, ChromaDB, Tool Routing, RAG Pipeline
■ Week 3	Graph BFS Router, DFS Agent Loop, Multi-Tool Chain
■ Week 4	Bug Fixes, BFS Upgrade, DFS Agent, Multi-Chain (In Progress)

WEEK 1

Week 1 — DSA AI Tutor Chatbot v1

■ DSA: Arrays, HashMaps, Stacks, Queues, Linked Lists ■ AI: Groq API, Prompt Engineering, Chaining ■ Project: DSA AI Tutor Chatbot v1

W1 · D1 — Setup & First API Call ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Arrays & HashMaps

- Array = ordered list, $O(1)$ index access, $O(n)$ search — Python list
- HashMap = key-value store, $O(1)$ average lookup — Python dict
- Array stores things in order; HashMap finds things instantly by key

```
chat_history = [] # Array
cache = {} # HashMap
cache['what is array'] = 'ordered list' #  $O(1)$  insert
cache.get('what is array') #  $O(1)$  lookup
```

■ *Your chatbot needs to remember last 5 messages — Array or HashMap, and why?*
Think:

■ BLOCK — DSA Problem (30 min)

#1 — Two Sum

leetcode.com/problems/two-sum

Hint: Store each number in a HashMap as you scan; check if complement already exists — one pass, no nested loop

Why this problem: Forces you to think Array (brute force $O(n^2)$) vs HashMap (optimized $O(n)$) — exact same decision you make building chatbot cache

■ BLOCK — AI Concept (20 min)

Concept: Groq API + LLM Message Format

- Groq is a free LLM API — no credit card, fast response, same interface as OpenAI
- API takes a list of messages — that list IS an Array, each message IS a HashMap
- Sign up at groq.com → get free API key in 2 minutes

```
messages = [ # Array
  {'role': 'system', 'content': '...'}, # HashMap
  {'role': 'user', 'content': '...'}, # HashMap
]
```

■ BLOCK — AI Project (40 min)

What you're building: CLI chatbot using Arrays and HashMaps from Block 1

The idea:

user types → append to chat_history list (Array!) → check cache dict (HashMap!) → cached? → return instantly → not cached? → call Groq → store in cache → return reply

Your task:

1. Save as week1_day1.py
2. Get Groq API key → store in .env file
3. Build chat loop: input → store in list → call Groq → print reply
4. Add caching: same question twice → return cached answer, skip API call

Verify:

- Chatbot replies to any question
- Same question twice returns cached answer instantly
- chat_history list grows with each message
- API key not hardcoded in file

■ Reflect (5 min)

1. Where exactly did you use an Array in your code?
2. Where exactly did you use a HashMap?
3. What happens if cache grows too large — how would you fix it? ■

■ Tomorrow — Day 2 — Stacks & Queues → conversation memory with token limit

W1 · D2 — Conversation Memory ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Stacks & Queues

- Stack = LIFO (Last In First Out) — like a pile of plates, Python list with append/pop
- Queue = FIFO (First In First Out) — like a line at a store, Python deque
- Your chatbot needs a Queue for chat history — oldest messages leave first when full

```
from collections import deque
queue = deque(maxlen=10) # auto-drops oldest when full
queue.append('msg1')
queue.append('msg2')
# stack
stack = []
stack.append('step1')
stack.pop() # removes last
```

■ *Chat history has a token limit. Would Stack or Queue handle this better — which removes the right end?*
Think:

■ BLOCK — DSA Problem (30 min)

#20 — Valid Parentheses

leetcode.com/problems/valid-parentheses

Hint: Push opening brackets onto stack; when closing bracket found, pop and check if it matches

Why this problem: Classic Stack problem — same push/pop logic you use to track prompt chain steps in your chatbot

■ BLOCK — AI Concept (20 min)

Concept: Conversation History & Token Management

- LLMs have context limits — you can't send infinite history
- Solution: trim oldest messages when token count exceeds budget
- Token $\approx$ word/4 chars — count tokens before every API call

```
chat_history = deque() # Queue from DSA!
def trim_to_budget(history, max_tokens=3000):
    while token_count(history) > max_tokens:
        history.popleft() # remove oldest message
```

■ BLOCK — AI Project (40 min)

What you're building: Chatbot with smart memory that trims itself when it gets too long

The idea:

user types → append to deque (Queue!) → count total tokens in history → over limit? → popleft() oldest messages → send trimmed history to Groq → get reply → append to history

Your task:

1. Save as week1_day2.py
2. Replace plain list with deque for chat_history
3. Add token counting function (len(text)//4 is fine)
4. Add trim_to_budget() that pops oldest when over limit
5. Test: have a long conversation, verify old messages disappear

Verify:

- Long conversation doesn't crash or send too many tokens
- Oldest messages removed first (not newest)
- System prompt always stays — never trimmed
- Token count stays under your budget

■ Reflect (5 min)

1. Why deque instead of a plain list for chat history?
2. What happens if you pop from the wrong end?
3. Should the system prompt ever be trimmed — how do you protect it? ■

■ Tomorrow — Day 3 — Linked Lists → build a prompt chain that links AI calls together

W1 · D3 — Prompt Chaining ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Linked Lists

- Linked List = chain of nodes, each pointing to next — no random access, only sequential
- Each node has data + a pointer to next node
- Your prompt chain IS a linked list — each LLM response feeds into the next call

```
class Node:
def __init__(self, data):
self.data = data
self.next = None
# Chain: question → answer → refine → final
node1 = Node('raw answer')
node2 = Node('refined answer')
node1.next = node2
```

■ **Think:** *In your prompt chain, if step 2 fails should you retry from step 1 or skip to step 3? How does linked list help decide?*

■ BLOCK — DSA Problem (30 min)

#206 — Reverse Linked List

leetcode.com/problems/reverse-linked-list

Hint: Use three pointers: prev, curr, next — update one at a time as you traverse

Why this problem: Forces you to think about pointer manipulation — same concept as chaining LLM calls where output of one becomes input of next

■ BLOCK — AI Concept (20 min)

Concept: Prompt Chaining (Multi-step LLM Calls)

- Instead of one LLM call, chain multiple — output of call 1 becomes input of call 2
- Improves quality: first call generates, second call refines
- Use a stack to track each step — so you can debug or rollback

```
stack = [] # track chain steps
# Step 1: generate
r1 = ollama.chat(model=model, messages=[{'role':'user', 'content':prompt}])
base = r1['message']['content']
stack.append(base)
# Step 2: refine
r2 = ollama.chat(model=model, messages=[{'role':'user', 'content':f'refine: {base}'}])
refined = r2['message']['content']
stack.append(refined)
```

■ BLOCK — AI Project (40 min)

What you're building: Two-step prompt chain where first call generates answer, second call refines it

The idea:

user question → build prompt with context → call Groq → get base answer → push to stack → call Groq again with 'refine this: {base}' → check refined answer quality → if bad → pop stack → use base answer instead → return best answer

Your task:

1. Save as week1_day3.py
2. Add stack = [] to ChatMemory
3. In chain(): call Groq twice — generate then refine
4. Push each result to stack
5. If refined is too short or has 'error' → pop and use base

Verify:

- Two API calls happen per question
- Stack has 2 items after each response
- Bad refinement falls back to base answer
- Final answer is better quality than single call

■ Reflect (5 min)

1. When would chaining 3 calls be better than 2?
2. What's the cost of chaining — tokens, latency?
3. Could you chain in a loop until quality is good enough — what's the risk? ■

■ Tomorrow — Week 2, Day 1 — Embeddings & Semantic Search → your chatbot finds relevant info

WEEK 2

Week 2 — RAG Pipeline with Tool Router

■ DSA: Searching, Hash Tables, Sorting ■ AI: Embeddings, ChromaDB, Tool Routing ■ Project: RAG Pipeline with Tool Router

W2 · D1 — Embeddings & Semantic Search ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Searching & Binary Search

- Linear search = check every item $O(n)$ — slow for large data
- Binary search = split in half each time $O(\log n)$ — requires sorted data
- Semantic search is smarter: finds meaning, not just exact keyword match

```
# Binary search —  $O(\log n)$ 
def binary_search(arr, target):
    lo, hi = 0, len(arr)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid+1
        else: hi = mid-1
    return -1
```

■ **Think:** *Binary search needs sorted data. How would you 'sort' documents for semantic search — what does sorted even mean for text?*

■ BLOCK — DSA Problem (30 min)

#704 — Binary Search

leetcode.com/problems/binary-search

Hint: Always update lo or hi past mid — never set lo=mid or you risk infinite loop

Why this problem: Core search pattern — understanding $O(\log n)$ vs $O(n)$ is exactly why vector search beats keyword scanning in RAG

■ BLOCK — AI Concept (20 min)

Concept: Embeddings & Vector Similarity

- Embedding = converting text into a list of numbers (vector) that captures meaning
- Similar meaning → vectors are close together in space (high cosine similarity)
- Ollama can generate embeddings locally for free: `ollama.embeddings()`

```
import ollama
response = ollama.embeddings(model='qwen2.5', prompt='what is a graph?')
vector = response['embedding'] # list of ~4096 floats
# similar questions → similar vectors → can find related content
```


■ BLOCK — AI Project (40 min)

What you're building: Function that takes user question, embeds it, finds most similar document chunk

The idea:

load document text → split into chunks (size=200, overlap=50) → embed each chunk → store vectors → user asks question → embed question → compare to all chunk vectors → return top 3 most similar chunks

Your task:

1. Save as week2_day1.py
2. Write `chunk_text(text, size=200, overlap=50)`
3. Write `embed(text)` using `ollama.embeddings()`
4. Write `cosine_similarity(v1, v2)`
5. Write `find_top_k(query, chunks, k=3)` using your similarity function

Verify:

- Chunks overlap correctly — no content lost
- Embedding returns a list of floats
- Similar questions return similar chunks
- Unrelated questions return low similarity scores

■ Reflect (5 min)

1. Why overlap chunks instead of clean splits?
2. What happens if chunk size is too small? Too large?
3. Cosine similarity vs dot product — when does it matter? ■

■ Tomorrow — Day 2 — ChromaDB → store and retrieve embeddings with a real vector database

W2 · D2 — ChromaDB Vector Store ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Hash Tables (Deep Dive)

- Hash Table is what powers ChromaDB's fast retrieval under the hood
- Hashing maps a key to a fixed-size index — $O(1)$ average lookup
- Collision handling (chaining/open addressing) keeps it reliable

```
# Python dict is a hash table
d = {}
d['graph'] = [0.1, 0.4, ...] # hash('graph') → bucket → store
d['graph'] # hash('graph') → same bucket →  $O(1)$  get
```

■ **Think:** *ChromaDB stores millions of vectors. Why is hashing better than scanning all vectors linearly for every query?*

■ BLOCK — DSA Problem (30 min)

#146 — LRU Cache

leetcode.com/problems/lru-cache

Hint: Use OrderedDict — move accessed items to end, pop from front when over capacity

Why this problem: LRU = Least Recently Used — same eviction strategy your chatbot uses when `response_cache` is full (OrderedDict)

■ BLOCK — AI Concept (20 min)

Concept: ChromaDB — Persistent Vector Database

- ChromaDB stores embeddings on disk — persists between runs
- `get_or_create_collection` — safely loads existing or makes new
- Query returns top-k most similar documents automatically

```
import chromadb
from chromadb.config import Settings
client = chromadb.Client(Settings(persist_directory='./vectorstore'))
col = client.get_or_create_collection('dsa_docs')
col.add(documents=chunks, embeddings=embeddings, ids=ids)
result = col.query(query_embeddings=[q_vec], n_results=3)
top_chunks = result['documents'][0]
```

■ BLOCK — AI Project (40 min)

What you're building: `VectorStore` class that saves chunks to ChromaDB and retrieves relevant ones

The idea:

load document → create chunks → embed all chunks → store in ChromaDB (persists to disk) → on next run: skip embedding — load existing → user queries → embed query → ChromaDB returns top 3

Your task:

1. Save as week2_day2.py
2. Build VectorStore class with `__init__`, `add_chunks`, `search` methods
3. In `add_chunks`: check if collection already has data — skip if yes (`Loading_existing`)
4. In `search`: query ChromaDB and return top documents
5. Test: run twice — second run should print 'Loading_existing'

Verify:

- First run builds and saves vector store
- Second run loads existing — no re-embedding
- Search returns relevant chunks for DSA questions
- Unrelated query returns low-relevance results

■ Reflect (5 min)

1. Why check `collection.count()` before adding chunks?
2. What breaks if you re-add same chunks every run?
3. When would you want to rebuild the vector store from scratch? ■

■ Tomorrow — Day 3 — Tool Routing → chatbot decides which tool to use based on user input

W2 · D3 — Tool Routing & RAG Pipeline ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Sorting & Priority

- Sorting = arranging data by a key — $O(n \log n)$ for most algorithms
- Priority Queue = always gives you the highest priority item first
- RAG ranking = sort retrieved chunks by relevance score before using them

```
# Sort chunks by relevance score
chunks = [('graph def', 0.9), ('array def', 0.4), ('tree def', 0.7)]
chunks.sort(key=lambda x: x[1], reverse=True)
# → [('graph def', 0.9), ('tree def', 0.7), ('array def', 0.4)]
```

■ **Think:** *Your route_tool checks conditions in a fixed order. What if two conditions match — which wins, and is that the right behavior?*

■ BLOCK — DSA Problem (30 min)

#347 — Top K Frequent Elements

leetcode.com/problems/top-k-frequent-elements

Hint: Use a HashMap for frequency count, then heap or sort to get top k

Why this problem: Finding top-k by frequency = same logic as finding top-k relevant chunks by similarity score in RAG

■ BLOCK — AI Concept (20 min)

Concept: Tool Routing — Agent Decision Making

- Instead of always calling LLM, route to the right tool first
- Math question → calculator, Definition question → dict, History → memory, else → RAG
- Tool router is if/elif logic — Week 4 upgrades it to a graph

```
def route_tool(user_input, memory):
    if '**' in user_input or any(s in user_input for s in '+-*/'):
        return 'calculate', calculate(user_input)
    if any(k in user_input.lower() for k in ['what is', 'define']):
        return 'define', define_term(user_input)
    if any(k in user_input.lower() for k in ['history', 'discuss']):
        return 'history', get_history(memory)
    return 'search', search_document(user_input, memory)
```

■ BLOCK — AI Project (40 min)

What you're building: Full RAG pipeline: route → retrieve → prompt → chain → reply

The idea:

user question → route_tool() decides: calculate / define / history / search → if search: embed query → ChromaDB → top 3 chunks → build prompt: context + question → call Groq (step 1: generate, step 2: refine) → return refined answer

Your task:

1. Save as week2_day3.py
2. Combine VectorStore + ChatMemory + route_tool into one file
3. Add define_term dict with 5 DSA terms
4. Add calculate() using eval with safety check
5. Test all 4 routes: math, define, history, RAG search

Verify:

- '2**10' → 1024 (calculate route)
- 'what is graph' → definition (define route)
- 'what we discussed' → history (history route)
- 'explain binary search' → RAG search → LLM answer

■ Reflect (5 min)

1. Which route gets triggered most in testing?
2. What happens when route_tool picks the wrong tool?
3. How would you add a 5th tool — what's the cleanest way? ■

■ Tomorrow — Week 3, Day 1 — Graph theory → redesign router as a graph of decisions

WEEK 3

Week 3 — Graph-Routed Agent with Fallback

■ DSA: Graphs BFS/DFS, DP Preview ■ AI: BFS Router, Agent Loop, Multi-Chain ■ Project: Graph-Routed Agent with Fallback

W3 · D1 — Graph Theory & Router as a Graph ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Graphs — Nodes, Edges, Traversal

- Graph = set of nodes connected by edges — can be directed or undirected
- Adjacency list = HashMap where each node maps to list of neighbors
- BFS (Breadth First) = visit level by level using Queue; DFS (Depth First) = go deep using Stack/recursion

```
TOOL_GRAPH = {
  'start': ['math_check', 'keyword_check', 'history_check', 'default'],
  'math_check': ['calculate'],
  'keyword_check': ['define'],
  'history_check': ['history'],
  'default': ['search']
}
# BFS traversal
from collections import deque
queue = deque(TOOL_GRAPH['start'])
```

■ *Your current if/elif router is linear. What breaks if you add 10 more tools — and how does a graph fix that?*
Think:

■ BLOCK — DSA Problem (30 min)

#200 — Number of Islands

leetcode.com/problems/number-of-islands

Hint: Use BFS from each unvisited '1' cell — mark visited as '0' to avoid revisiting

Why this problem: BFS/DFS on a grid — exact same traversal logic you use to walk your tool routing graph

■ BLOCK — AI Concept (20 min)

Concept: BFS-Based Tool Router

- Replace if/elif chain with a graph of tool nodes
- BFS traverses graph — checks each node in order, stops when a tool matches
- Adding new tools = add a node + edge to graph, no touching existing logic

```

from collections import deque
def route_tool(user_input, memory):
    queue = deque(TOOL_GRAPH['start'])
    visited = set()
    while queue:
        node = queue.popleft()
        if node in visited: continue
        visited.add(node)
        name, result = check_node(node, user_input, memory)
        if name: return name, result
        for child in TOOL_GRAPH.get(node, []):
            if child not in visited: queue.append(child)
    return 'search', search_document(user_input, memory)

```

■ BLOCK — AI Project (40 min)

What you're building: Replace your if/elif route_tool with BFS graph traversal

The idea:

define TOOL_GRAPH as adjacency list → BFS starts at 'start' node → dequeue node → check_node() → does user input match? → yes → return tool result → no → add children to queue → continue BFS → fallback to search if nothing matched

Your task:

1. Save as week3_day1.py
2. Define TOOL_GRAPH dict (adjacency list)
3. Write check_node(node, user_input, memory) with conditions per node
4. Replace old route_tool with BFS version
5. Verify all 4 routes still work exactly as before

Verify:

- All 4 routes still work (math, define, history, search)
- Adding a new tool only requires adding to TOOL_GRAPH — not touching other code
- Visited set prevents infinite loops
- BFS order matches expected priority

■ Reflect (5 min)

1. Did BFS change which tool gets priority vs old if/elif?
2. What if two nodes both match — which wins in BFS?
3. When would DFS routing be better than BFS? ■

■ Tomorrow — Day 2 — DFS Agent Loop → chatbot retries with fallback tools when first result is weak

W3 · D2 — DFS Agent Loop ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: DFS — Depth First Search

- DFS = go deep into one path completely before trying another
- Uses a visited set to avoid revisiting — same as BFS but goes deep first
- In your agent: try one tool fully → if result weak → go deeper to next tool

```
def dfs(graph, start, visited=None):
    if visited is None: visited = set()
    visited.add(start)
    print(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

■ **Think:** *Your agent tries tools one by one. When should it stop retrying — after 2 failures? 3? What's the right max depth?*

■ BLOCK — DSA Problem (30 min)

#133 — Clone Graph

leetcode.com/problems/clone-graph

Hint: Use a HashMap {original_node: cloned_node} as visited — if already cloned, return it directly

Why this problem: DFS with visited dict — same structure as your agent loop's visited_tools set that prevents calling the same tool twice

■ BLOCK — AI Concept (20 min)

Concept: Agent Loop — Self-Deciding Fallback

- Instead of one tool call → stop, agent loops until it gets a good enough result
- Checks result quality after each tool — if weak, tries next tool as fallback
- Max depth prevents infinite loop — agent gives up after 3 tries

```
def agent_loop(self, user_, route_tool, max_depth=3):
    visited_tools = set() # DFS visited!
    depth = 0
    while depth < max_depth:
        tool_name, result = route_tool(user_, self)
        if tool_name in visited_tools: break
        visited_tools.add(tool_name)
        depth += 1
        if self._is_good_result(result): return tool_name, result
        # weak result → try search as fallback
    return tool_name, result
```

■ BLOCK — AI Project (40 min)

What you're building: Agent loop with DFS fallback — tries harder before giving up

The idea:

call route_tool → check result quality (_is_good_result) → good → return immediately → weak → add to visited_tools
→ try search fallback → search also weak → return honestly 'topic not in document' → max depth hit → stop

Your task:

1. Save as week3_day2.py
2. Add _is_good_result(result) that checks length, 'not found' phrases
3. Add agent_loop() with visited set and max_depth
4. Update chain() to call agent_loop instead of single route_tool
5. Test: 'what is binary search' → not in definitions → should fall to RAG

Verify:

- 'what is graph' → define route, returns immediately (1 iteration)
- 'what is binary search' → define fails → falls to RAG (2 iterations)
- 'explain quantum physics' → RAG weak → returns honest message
- Stack has correct number of entries after each chain

■ Reflect (5 min)

1. How many iterations did your agent use for each test case?
2. Did max_depth=3 ever get hit — what triggered it?
3. Should calculate ever trigger fallback — why or why not? ■

■ Tomorrow — Day 3 — Multi-tool chaining → one question triggers two tools, answers combined

W3 · D3 — Multi-Tool Chaining ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Dynamic Programming — Memoization Preview

- DP = breaking problem into subproblems, solving each once, storing results
- Memoization = cache subproblem results — never solve same thing twice
- Your `response_cache` IS memoization — same question never hits API twice

```
from functools import lru_cache
@lru_cache(maxsize=128)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
# Without cache:  $O(2^n)$  With cache:  $O(n)$ 
```

■ **Think:** *Your chatbot caches full question→answer pairs. Could you cache partial results (just the retrieved chunks) separately — what's the benefit?*

■ BLOCK — DSA Problem (30 min)

#70 — Climbing Stairs

leetcode.com/problems/climbing-stairs

Hint: $fib(n) = fib(n-1) + fib(n-2)$ — store results in a dict, check before computing

Why this problem: Classic memoization intro — teaches you to recognize overlapping subproblems, same pattern as caching LLM responses

■ BLOCK — AI Concept (20 min)

Concept: Multi-Tool Chaining — Combining Results

- One question can benefit from multiple tools — e.g. define + search + refine
- Chain results: tool1 output → feeds into tool2 prompt → final answer
- Different from agent loop: both tools run intentionally, not as fallback

```
def multi_chain(self, user_, tools):
    results = []
    for tool_name in tools:
        _, output = TOOLS[tool_name](user_, self)
        if self._is_good_result(output):
            results.append(output)
    combined = ' '.join(results)
    # send combined context to LLM for final answer
    return self._llm_synthesize(combined, user_)
```

■ BLOCK — AI Project (40 min)

What you're building: Chatbot that combines define + RAG search results into one richer answer

The idea:

user asks question → check define_term dict → get short definition if exists → embed query → ChromaDB → get top 3 chunks → combine: definition + chunks as context → send to Groq: 'using this context, answer the question fully' → return synthesized answer

Your task:

1. Save as week3_day3.py
2. Add multi_chain() method that runs define + search sequentially
3. Combine results into one context string
4. Update chain() to use multi_chain when both tools return results
5. Test: 'explain graph traversal' → should use both define + RAG

Verify:

- 'what is graph' → gets definition AND document context
- Single tool question still works normally
- Combined context produces richer answer than either alone
- No duplicate content in combined result

■ Reflect (5 min)

1. Did combining tools actually improve answer quality?
2. What's the token cost of combining vs single tool?
3. When would you NOT want to combine — what's a bad combo? ■

■ Tomorrow — Week 4, Day 1 — Full project review + bug fixes before moving to production features

WEEK 4

Week 4 — Production-Ready Chatbot

■ DSA: Graph BFS/DFS Deep Dive, DP ■ AI: Bug Fixes, Upgraded Router, Agent, Synthesis ■ Project: Production-Ready Chatbot

W4 · D1 — Project Review & Bug Fixes ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Recap — All Structures Used So Far

- Array (list) → chat_history stores messages in order
- HashMap (dict/OrderedDict) → response_cache for O(1) lookup
- Queue (deque) → chat_history with token trimming (FIFO)
- Stack (list) → prompt chain tracking (LIFO)
- Graph (dict of lists) → tool routing via BFS

```
# Your project uses ALL of these:
chat_history = deque() # Queue
response_cache = OrderedDict() # HashMap (LRU)
stack = [] # Stack
TOOL_GRAPH = {} # Graph
embeddings = [] # Array
```

■ **Think:** Looking at your full codebase — which data structure is doing the most work? Which one could you remove without breaking anything?

■ BLOCK — DSA Problem (30 min)

#242 — Valid Anagram

leetcode.com/problems/valid-anagram

Hint: Count character frequencies for both strings using Counter or a dict — compare the two dicts

Why this problem: Revisit HashMap frequency counting — reinforces the pattern you use in token counting and cache key matching

■ BLOCK — AI Concept (20 min)

Concept: Code Quality — Normalize, Cache Keys, Edge Cases

- normalize() cleans user input — removes punctuation, lowercases — for consistent cache keys
- Bug: normalizing before routing vs after causes mismatches in history and cache
- Fix: normalize only for cache key, store original text in chat_history

```
# WRONG: normalize before everything
normalized = self.normalize(user_)
self.chat_history.append({'role': 'user', 'content': normalized})
# RIGHT: normalize only for cache key
self.chat_history.append({'role': 'user', 'content': user_}) # original
cache_key = self.normalize(user_) # normalized for cache only
```

■ BLOCK — AI Project (40 min)

What you're building: Audit and fix all known bugs in your current codebase

The idea:

run chatbot with 5 test inputs → check: history shows correct messages → check: cache hit on repeated questions → check: new document content gets indexed → check: get_history returns full conversation → fix each bug found

Your task:

1. Save as week4_day1.py (clean version)
2. Fix normalize bug: store original in history, normalize only for cache
3. Fix get_history: show all messages not just last 3
4. Fix ChromaDB: add new chunks that don't already exist (check IDs)
5. Run all 4 route tests and confirm clean output

Verify:

- 'what we discuss' shows full conversation history
- Same question twice → cache hit message printed
- New content in document file gets picked up on next run
- No duplicate chunks in ChromaDB

■ Reflect (5 min)

1. Which bug was hardest to find and why?
2. How would you write a test to catch the normalize bug automatically?
3. What would break first if you scaled this to 10,000 documents? ■

■ Tomorrow — Day 2 — Graph BFS router upgrade → cleaner, extensible tool routing

W4 · D2 — BFS Router Upgrade ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: BFS Deep Dive — Level Order Traversal

- BFS visits all nodes at current level before going deeper
- Uses a Queue — enqueue neighbors, dequeue and process one by one
- Level order = first all direct neighbors, then their neighbors

```
from collections import deque
def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

■ **Think:** *In your TOOL_GRAPH, 'start' has 4 neighbors. BFS visits them all before going deeper. Is that the right priority order for your chatbot?*

■ BLOCK — DSA Problem (30 min)

#102 — Binary Tree Level Order Traversal

leetcode.com/problems/binary-tree-level-order-traversal

Hint: Use a deque, enqueue root, then for each level enqueue all children before processing next level

Why this problem: Classic BFS on tree — same Queue pattern you use to traverse TOOL_GRAPH level by level

■ BLOCK — AI Concept (20 min)

Concept: Extensible Graph Router with check_node

- Each node in TOOL_GRAPH has a matching check_node() condition
- Adding new tool = add node to graph + add condition to check_node — no other changes
- This is the Open/Closed principle: open for extension, closed for modification

```
def check_node(node, user_input, memory):
    u = user_input.lower()
    if node == 'math_check':
        if '**' in user_input or any(s in user_input for s in '+-*/'):
            return 'calculate', TOOLS['calculate'](user_input)
    if node == 'keyword_check':
        if any(k in u for k in ['what is', 'define']):
            term = u.replace('what is', '').replace('define', '').strip()
            return 'define', TOOLS['define'](term)
    # ... other nodes
    return None, None
```

■ BLOCK — AI Project (40 min)

What you're building: Clean, extensible BFS router replacing old if/elif chain

The idea:

define TOOL_GRAPH with all nodes → BFS starts at 'start' → dequeue node → check_node() matches? → yes → return result → no → add children to queue → repeat until queue empty → fallback to search

Your task:

1. Save as week4_day2.py
2. Define complete TOOL_GRAPH with 5 nodes
3. Write check_node() covering all conditions
4. Implement BFS route_tool using deque + visited set
5. Add a 6th tool 'summarize' — only add to graph + check_node

Verify:

- All existing routes still pass
- New 'summarize' tool works when triggered
- Adding summarize required no changes to BFS loop itself
- Visited set prevents any node from running twice

■ Reflect (5 min)

1. Did the graph structure make adding 'summarize' easier than old if/elif?
2. What's the time complexity of your BFS router?
3. How would you make TOOL_GRAPH configurable from a JSON file? ■

■ Tomorrow — Day 3 — DFS Agent Loop with fallback → chatbot retries smarter

W4 · D3 — DFS Agent Loop with Smart Fallback ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: DFS vs BFS — When to Use Which

- BFS = good for finding shortest path, level-by-level exploration (your router)
- DFS = good for exploring one path fully before backtracking (your agent loop)
- Agent loop uses DFS: try tool → weak? → go deeper into fallback → weak? → give up

```
# DFS iterative using explicit stack
def dfs_iterative(graph, start):
    stack = [start]
    visited = set()
    while stack:
        node = stack.pop() # LIFO — go deep
        if node in visited: continue
        visited.add(node)
        print(node)
        for neighbor in graph[node]:
            stack.append(neighbor)
```

■ **Think:** *BFS router gives first match quickly. DFS agent loop explores deeply. Could you combine them — BFS to pick tool, DFS to retry on failure?*

■ BLOCK — DSA Problem (30 min)

#133 — Clone Graph

leetcode.com/problems/clone-graph

Hint: visited dict maps original→clone; if node already in dict return clone directly without re-processing

Why this problem: DFS with visited HashMap — exact pattern of agent_loop visited_tools set preventing same tool from running twice

■ BLOCK — AI Concept (20 min)

Concept: Agent Loop — Quality Check & Fallback

- `_is_good_result()` is your quality gate — checks length, error phrases, 'not found'
- Agent tries tool → weak result → tries search → still weak → returns honest message
- Max depth=3 prevents runaway loops — agent gives up gracefully

```
def _is_good_result(self, result):
    if not result: return False
    bad = ['not found', 'no context', 'definition not found', 'error']
    if any(b in result.lower() for b in bad): return False
    if len(result.strip()) < 20: return False
    return True
```

■ BLOCK — AI Project (40 min)

What you're building: Agent loop that DFS-retries with fallback tools when first result is weak

The idea:

call BFS route_tool → get tool + result → `_is_good_result?` → yes → return → no → add to visited_tools → try search fallback → search result good? → return → both weak → return 'topic not covered in document' → max depth hit → stop regardless

Your task:

1. Save as week4_day3.py
2. Add `_is_good_result()` with all quality checks
3. Add `agent_loop()` with visited set and `max_depth=3`
4. Update `chain()` to call `agent_loop` instead of `route_tool` directly
5. Test: query not in document → should get honest fallback message

Verify:

- 'what is graph' → direct define result (1 iteration)
- 'explain merge sort' (not in doc) → define weak → search weak → honest message
- '2**8' → calculate returns immediately, no loop
- Stack entries reflect actual number of chain steps taken

■ Reflect (5 min)

1. How many iterations for each of your 4 test cases?
2. Did `max_depth` ever get hit — what caused it?
3. Is `_is_good_result()` too strict or too lenient — how would you tune it? ■

■ Tomorrow — Day 4 — Multi-tool chaining → combine define + RAG for richer answers

W4 · D4 — Multi-Tool Chaining & Combined Answers ■ 2 hours total

■ BLOCK — DSA Concept (30 min)

Concept: Dynamic Programming — Overlapping Subproblems

- DP = recognize subproblems that repeat → solve once → reuse answer
- Two properties needed: optimal substructure + overlapping subproblems
- Your multi-tool chain IS DP thinking: define gives part of answer, RAG gives rest — combine once

```
# Fibonacci with memoization
memo = {}
def fib(n):
    if n in memo: return memo[n] # reuse!
    if n <= 1: return n
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
#  $O(2^n)$  →  $O(n)$  with memoization
```

■ **Think:** *In multi-tool chaining, you call define + search for every question. Could you cache the 'define' result separately from 'search' — what's the benefit?*

■ BLOCK — DSA Problem (30 min)

#70 — Climbing Stairs

leetcode.com/problems/climbing-stairs

Hint: Ways to reach step n = $\text{ways}(n-1) + \text{ways}(n-2)$ — store each in a dict before recursing

Why this problem: Simplest DP problem — teaches memoization pattern you apply to caching partial results in multi-tool chain

■ BLOCK — AI Concept (20 min)

Concept: Synthesizing Multiple Tool Results

- Multi-chain: run define + search both → combine into one rich context
- Send combined context to LLM: 'using both sources, give a complete answer'
- Better than either alone: definition gives precision, RAG gives detail

```
def multi_chain(self, user_):
    results = []
    # Tool 1: define
    _, defn = TOOLS['define'](user_, self)
    if self._is_good_result(defn): results.append(f'Definition: {defn}')
    # Tool 2: RAG search
    context = self.retrieve_context(user_)
    if context != 'No Context Found': results.append(f'Document context: {context}')
    if not results: return 'Topic not covered in loaded document.'
    combined = ' '.join(results)
    prompt = f'Using this info: {combined} Answer: {user_}'
    r = ollama.chat(model=self.model, messages=[{'role': 'user', 'content': prompt}])
    return r['message']['content']
```

■ BLOCK — AI Project (40 min)

What you're building: Chatbot that combines define + RAG results into one synthesized answer

The idea:

user asks question → run `define_term` → good result? → add to results list → run `retrive_context` → found? → add to results list → combine results into one context string → send to Groq: 'using this info, answer fully' → return synthesized response

Your task:

1. Save as `week4_day4.py`
2. Add `multi_chain()` method to `ChatMemory`
3. Update `chain()` to call `multi_chain()` for search route
4. Keep `calculate/history` routes as direct returns — no `multi_chain`
5. Test: 'explain graph traversal' should use both define + RAG

Verify:

- 'explain graph' → gets definition + document context both used
- '2**8' → still returns 1024 directly without `multi_chain`
- 'what we discuss' → history still works directly
- Combined answer is visibly richer than single-tool answer

■ Reflect (5 min)

1. Did combining tools improve the answer quality — by how much?
2. What's the extra token cost of `multi_chain` vs single tool?
3. When would you NOT combine — give a specific example from your tests ■

■ Tomorrow — Day 5 — Production cleanup: logging, stats, clean CLI interface